

The background features a complex network of thin, light-colored lines and dots, forming various triangular shapes and a larger, interconnected web-like structure. The lines are primarily in shades of light green and yellow, while the dots are small, dark grey or black. The overall effect is a technical, network-like aesthetic.

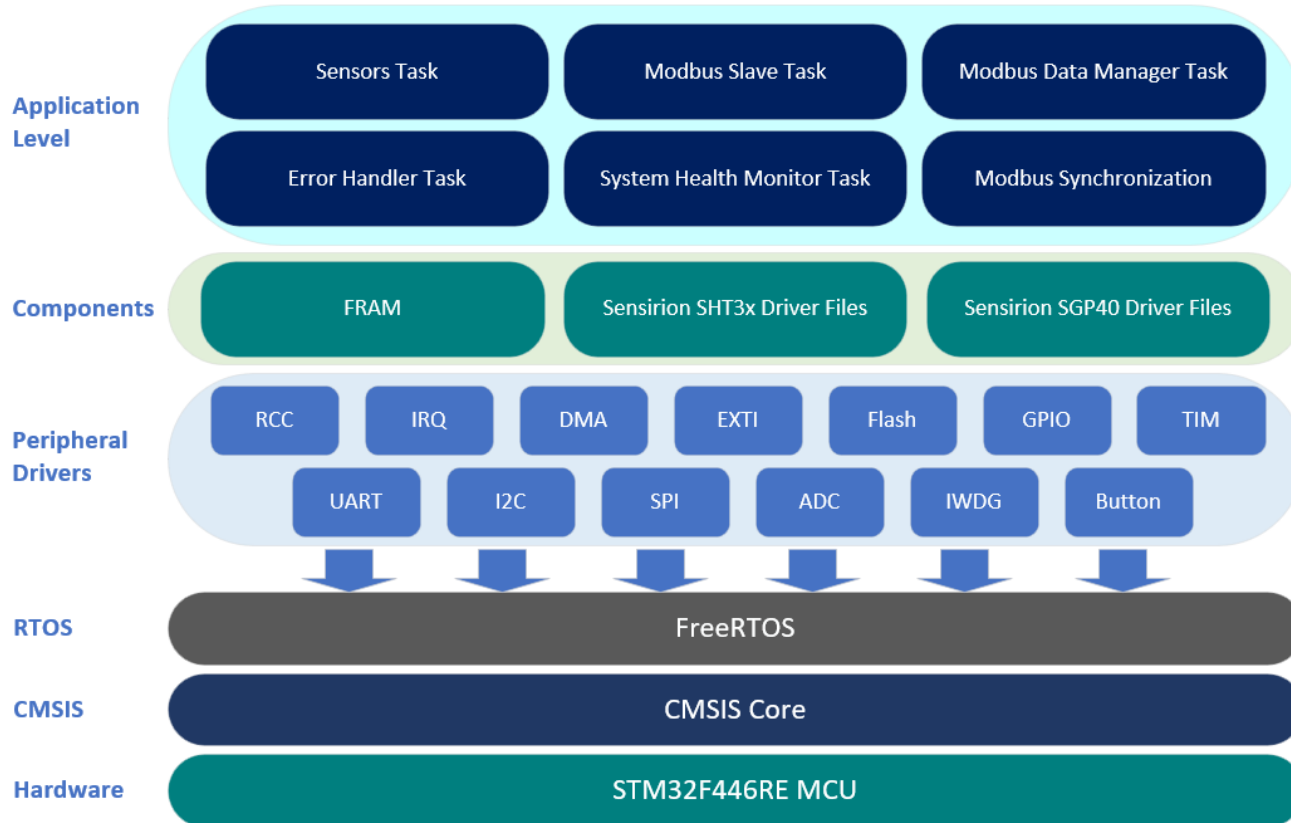
Project Setup Overview

Overview of CMSIS Core & FreeRTOS Project Setup

Project Setup Overview

- CMSIS Core Integration
- FreeRTOS Integration
- FreeRTOS Configuration
- FreeRTOS Startup Task Creation

Project Structure: CMSIS & FreeRTOS



CMSIS Core Integration

Overview

- **Objective:** Set up a CubeIDE project for STM32F446RE with CMSIS Core.
- **Key Steps:** Workspace creation, project initialization, file organization, and IDE configuration.
- **Basic Project Setup:** Enables register-level programming with CMSIS Core for efficient and low-level device control.
- **Organization of the CMSIS Core:** A clean, well-organized development environment ready for further development and integration with FreeRTOS.

FreeRTOS Integration

Overview

- **Objective:** Integrate FreeRTOS into the CubeIDE project for STM32.
- **Process Overview**
 - Creation of 'Middleware' and 'ThirdParty' directories for future scalability.
 - Import the FreeRTOS kernel source code and select only the necessary files.
 - Streamlining the 'Portable' directory for the ARM Cortex M4 processor.
 - Elimination of superfluous heap management schemes.
 - Updating the IDE's include paths to recognize FreeRTOS directories.
 - Incorporation of a pre-configured 'FreeRTOSConfig.h' from the STM32Cube Firmware package.
- **Outcome:** A CubeIDE project template configured with FreeRTOS, ready for real-time application development.

FreeRTOS Configuration

Overview

- **Update Core Clock Configuration**
 - Validate and update the 'SystemCoreClock' variable.
 - Utilize 'SystemCoreClockUpdate()' to ensure CPU clock speed for FreeRTOS.
- **Configure FreeRTOS Parameters**
 - Discuss preemptive scheduling and priority levels.
 - Adjust the heap size to match the application's memory requirements.
 - Enable mutexes and other synchronization primitives.
 - Ensure ISR priority levels are defined according to ARM Cortex-M and FreeRTOS guidelines to ensure system stability and real-time performance.
- **Outcome:** A well-configured 'FreeRTOSConfig.h', ensuring the system is tailored for the STM32 and ready for real-time tasks.

FreeRTOS Startup Task Creation

Overview

- **Objective:** Create a self-deleting FreeRTOS startup task for initial setup routines and configuration header file to support all task priority and stack size definitions.
- **Process Overview**
 - Create 'FreeRTOSTasks.h' file, including the necessary FreeRTOS headers.
 - Set task stack size and priority based on system requirements.
 - In 'main', define task prototypes, update SystemCoreClock, and kickstart the scheduler.
 - Code the startup task, then self-delete to free up memory.
- **Outcome:** Establish a startup task template for clean application bring-up.

The background features a complex network of thin, light-colored lines and small circles, forming a web-like structure. Overlaid on this are several translucent triangles of various sizes and orientations, some of which are filled with a very light yellow or green color. The overall aesthetic is technical and modern.

Next: CMSIS Core Integration
